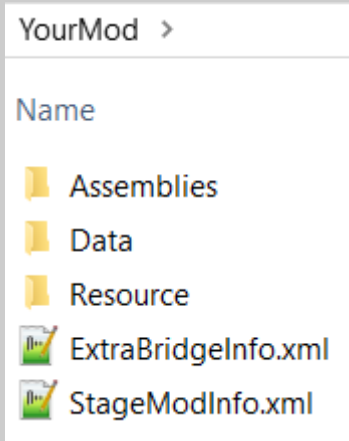
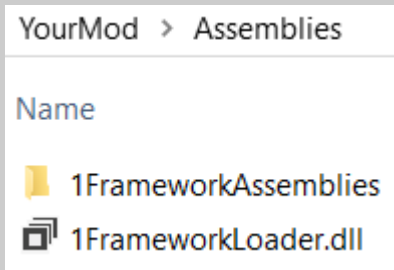


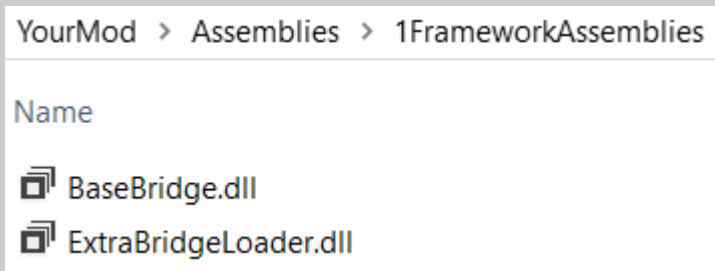
Basic setup



← put config file (see Configuration) into mod folder



put 1FrameworkLoader into Assemblies



then put bridge dlls into 1FrameworkAssemblies

Configuration

```
<?xml version="1.0" encoding="utf-8"?>
<ExtraBridgeInfo>
```

```
  <Gift Exist="true">
    <Path>\Data\GiftInfo.xml</Path>
  </Gift>
  <Formation Exist="true">
    <Path>\Data\FormationInfo.xml</Path>
  </Formation>
  <EmotionCard Exist="true">
    <Path>\Data\EmotionCard.xml</Path>
  </EmotionCard>
  <EmotionEgo Exist="true">
    <Path>\Data\EmotionEgo.xml</Path>
  </EmotionEgo>
```

```
  <HasCustomInnerType>true</HasCustomInnerType>
```

```
  <HasCustomCardData>true</HasCustomCardData>
```

```
  <HasCustomEquipData>true</HasCustomEquipData>
```

```
  <HasCustomOnlyCard>true</HasCustomOnlyCard>
```

```
  <GiftArtwork Exist="true">
    <Path>\Resource\GiftArtwork</Path>
  </GiftArtwork>
```

```
</ExtraBridgeInfo>
```

(any of the following 4 can be either a path to a single xml file, or a path to a directory containing xml files (possibly nested))

← Gift data path (see Gifts)

← Formation data path (see Formations)

← EmotionCard data path (see EmotionCards)

← EmotionEgo data path (see EmotionEgo)

← Enables loading CustomInnerType (see Extra)

← Enables loading CustomCardData (see Extra)

← Enables loading CustomEquipData (see Extra)

← Enables loading CustomOnlyCard (see Extra)

← Gift image path (see Gifts)

Gifts

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<GiftXmlRoot>
```

```
<Gift ID="1">
```

```
<Name>CustomGiftNameId</Name>
```

```
<Count>1</Count>
```

```
<Resource>ExtraBridge_ResName</Resource>
```

```
<IsAuraPreview>false</IsAuraPreview>
```

```
<Position>Eye</Position>
```

```
<NoAppear>false</NoAppear>
```

```
<PriorityOrder>Guest</PriorityOrder>
```

```
<Passive>1</Passive>
```

```
<CustomPassive>CustomPassive</CustomPassive>
```

```
<Stat>
```

```
<HP>0</HP>
```

```
<Break>0</Break>
```

```
</Stat>
```

```
</Gift>
```

```
</GiftXmlRoot>
```

numerical id; can be used to access the loaded gift in BaseBridge (using new Lorld(YourPackageld, ID))

ID for gift name/description/condition localization (using e.g. Localization Manager)

replaces {0} (if present) in gift's AcquireCondition text (defaults to 1)

resource name (used for creating gift's visual appearance; see below)

if true, the gift will have the aura preview image (Book of Distortion icon)

gift slot (Eye/Nose/Cheek/Mouth/Ear/HairAccessory/Hood/Mask/Helmet/None)

if true, the gift cannot be equipped and only provides title options (defaults to false)

priority for roughly ordering gifts by "category" in equipment/title menus (Sephirah before Librarian before Guest before Creature; defaults to Guest)

give GiftPassiveAbility_1 in combat (only vanilla; optional, can be multiple)

give PassiveAbility_CustomPassive in combat (optional, can be multiple)

bonus stat block (HP = HP bonus, Break = Stagger Resist bonus; both can be negative, both can be omitted, the block itself can be omitted)

Gifts: resource names

To use simple custom gift appearance, give your gift a resource name of ExtraBridge_{ResName}, where {ResName} is an arbitrary name (try to keep it distinct to avoid accidental conflicts), and place files named {ResName}_Front.png, {ResName}_Front_Back.png, {ResName}_Side.png and/or {ResName}_Side_Back.png (first two are used for default view, second two for side view; any can be omitted if not necessary) into the GiftArtwork folder specified in your config.

To use custom gift appearances created via your own code, check the dll documentation for ExtraBridgeUtils.customGiftGenerators (note that gift menu previews will still use simple sprites, so prepare at least one sprite accordingly or use IsAuraPreview).

Formations

```
<?xml version="1.0" encoding="utf-8" ?>
<FormationXmlRoot>

  <Formation ID="1">
    <Name>my formation</Name>
    <Position>
      <Name>my position 1</Name>
      <Vector x="12" y="0"/>
    </Position>
    <Position>
      <Name>my position 2</Name>
      <Vector x="6" y="-12"/>
    </Position>
    <Position>
      <Name>my position 3</Name>
      <Vector x="5" y="12"/>
    </Position>
    <Position>
      <Name>my position 4</Name>
      <Vector x="18" y="12"/>
    </Position>
    <Position>
      <Name>my position 5</Name>
      <Vector x="17" y="-12"/>
    </Position>
  </Formation>

</FormationXmlRoot>
```

numerical id; can be used to access the loaded formation in BaseBridge (using new LorId(YourPackageId, ID))

list of position coordinates

can have less or more than 5 positions, depending on your needs

position names are not important to the game, neither is the name of the formation itself; you can use them to help yourself remember what's intended for what, or you can even omit them entirely

the x coordinate is horizontal displacement towards the "associated side" (right for librarians in normal fights OR enemies in abnormality fights, left for vice versa), the y coordinate is displacement "upwards" (actually away from screen but it appears upwards)

the center of the map is (0, 0) regardless of faction

the given example is the default librarian formation for reference

using in stage xml:

```
<Wave>
  <!-- stuff -->
  <Formation>1</Formation>
  <CustomFormation>1</CustomFormation>
  <!-- other stuff -->
</Wave>
```

use CustomFormation instead of Formation

can also use <CustomFormation Pid="otherpackageid"> to use a formation from another BaseBridge-integrated mod (if it is loaded)

EmotionCards

```
<?xml version="1.0" encoding="utf-8"?>
<EmotionCardXmlRoot>
```

```
  <EmotionCard ID="1">
    <Name>CustomEmotionCardDescId</Name>
    <State>Negative</State>
    <Sephirah>ETC</Sephirah>
    <Level>0</Level>
    <EmotionLevel>0</EmotionLevel>
    <EmotionRate>0</EmotionRate>
    <Artwork>CombatPageArtwork.png</Artwork>
    <TargetType>SelectOne</TargetType>
    <Script>Custom, MyAssembly</Script>
    <Gift>1</Gift>
  </EmotionCard>
</EmotionCardXmlRoot>
```

numerical id; can be used to access the loaded card in BaseBridge (using new Lorld(YourPackageId, ID))

ID for card name/description/flavor text localization (using e.g. Localization Manager)
Positive/Negative

floor SephirahType (can be custom, e.g. Daat); note that setting this to anything but None or ETC will make your card available for picking naturally on the corresponding floor (if level/rate conditions are met)!

floor level required to be able to obtain the card (unused for None/ETC cards)

emotion level at which the card can be obtained (unused for None/ETC cards);
note that 1 stands for 1-2, 2 for 3-4, and 3 for 5+!

emotion rate (unused for None/ETC; between 0 and 2 for Positive, between 0 and -2 for Negative, absolute value roughly standing for "emotion coin imbalance bias")

card image (must be either a vanilla EmotionCard image OR be in the mod's CombatPageArtwork)

SelectOne/All/AllIncludingEnemy - note that the last one does NOT normally work for naturally picked enemy cards, and will result in not being applied to anyone!

EmotionCardAbility_Custom in MyAssembly dll; note that what matters here is the QUALIFIED class name (namespace and assembly included!), like so:

```
//no namespace
class EmotionCardAbility_Custom : EmotionCardAbilityBase
```

→ Custom, MyAssembly

```
namespace DoesNotStartWithEmotionCardAbility_
```

```
{
  class EmotionCardAbility_Custom : EmotionCardAbilityBase
```

→ WILL NOT WORK

```
namespace EmotionCardAbility_something
```

```
{
  class CustomAbility : EmotionCardAbilityBase
```

→ something.CustomAbility, MyAssembly

gift obtained by using the card 10 times (optional, can be multiple;
can reference gifts from other BaseBridge-integrated mods by
using <Gift Pid="otherpackageid">, or vanilla/fixed id gifts by
using <Gift Pid="@origin">)

xml id integration:

use CustomEmotionCard to add a card to the list of randomly pickable by enemies in a wave (Sephirah must be None; can be multiple or reference cards from other BaseBridge-integrated mods or vanilla by specifying Pid)

```
<Wave>
  <!-- stuff -->
  <CustomEmotionCard>1</CustomEmotionCard>
  <!-- other stuff -->
</Wave>
```

EmotionEgo

```
<?xml version="1.0" encoding="utf-8"?>
<EmotionEgoXmlRoot>

  <EmotionEgo ID="1"> ←
    <Sephirah>None</Sephirah> ←
    <Card>1</Card> ←
    <Gift>1</Gift> ←
  </EmotionEgo>

</EmotionEgoXmlRoot>
```

numerical id; can be used to access the loaded data in BaseBridge (using new LorId(YourPackageId, ID))

floor SephirahType (can be custom, e.g. Daat); note that setting this to anything but None or ETC will make the corresponding card available for picking as floor EGO on the corresponding floor (if floor EGO selection is available)!

corresponding card id (can include Pid to reference cards from other mods or vanilla)

gift obtained by using the corresponding card 10 times (same as in EmotionCard); note that the gift acquisition count is only performed for cards that are also marked as EGO in the card info (including EgoPersonal)

Extra: CustomInnerType

(requires HasCustomInnerType to be set to true in the config)

Inner type = "can not overlap" passive category (at most one per passive); passives with the same inner type cannot be attributed if one is already present.

For example, the standard inner type for "Speed" passives is 1.

```
<Passive ID="1">
  <!-- stuff -->
  <del>InnerType>1</InnerType>
  <CustomInnerType>myinnertype</CustomInnerType>
  <!-- other stuff -->
</Passive>
```

(in passive xml)

```
<Passive ID="1">
  <!-- stuff -->
  <del>InnerType>1</InnerType>
  <CopyInnerTypeFrom Pid="@origin">250023</CopyInnerTypeFrom>
  <!-- other stuff -->
</Passive>
```

To use CustomInnerType, specify it as a name; all passives with the same CustomInnerType will be given the same type that is distinct from others. This is recommended to use for making categories that are completely new.

To use CopyInnerTypeFrom, specify it as a reference to another passive (Pid available); the passive will be given the same inner type as the referenced one (if the referenced passive does not have an inner type yet, it will also be given a distinct one before copying). For example, this will make the passive unable to overlap with "The Commander".

Extra: CustomOnlyCard

(also known as exclusive pages)

(requires HasCustomOnlyCard OR HasCustomEquipData to be set to true in the config)

(in equip xml)

```
<EquipEffect>
  <!-- stuff -->
  <del><OnlyCard>1</OnlyCard></del>
  <CustomOnlyCard>1</CustomOnlyCard>
  <!-- other stuff -->
</EquipEffect>
```

use CustomOnlyCard instead of OnlyCard to specify cards from your mod; can specify Pid to use cards from other mods

(requires the card itself to also be marked as OnlyPage

in card xml as follows:

```
<Card ID="1">
  <!-- stuff -->
  <Option>OnlyPage</Option> )
  <!-- other stuff -->
</Card>
```

Extra: CustomEquipData

(requires HasCustomEquipData to be set to true in the config)

(in equip xml)

```
<Book ID="1">
  <!-- stuff -->
  <CustomOption>MyCustomOption</CustomOption>
  <CustomCategory>MyCustomCategory</CustomCategory>
  <!-- other stuff -->
</Book>
```

use CustomOption or CustomCategory to put custom values into equip's .optionList or .categoryList (each accepts multiple values) (use ExtraBridgeTools.MakeEnum to identify added enum values by their type and name)

Extra: CustomCardData

(requires HasCustomCardData to be set to true in the config)

(in card xml)

```
<Card ID="1">
  <!-- stuff -->
  <CustomOption>MyCustomOption</Option>
  <CustomCategory>MyCustomCategory</CustomCategory>
  <!-- other stuff -->
</Card>
```

use CustomOption or CustomCategory to put custom values into card's .optionList (accepts multiple values) or .category (only accepts a single value!)

(use ExtraBridgeTools.MakeEnum to identify added enum values by their type and name)

Extra: CustomStageFloor

(does not require configuration)

(in stage xml)

```
<Stage id="1">
  <!-- stuff -->
  <CustomExceptFloor>Daat</CustomExceptFloor>
  <CustomFloorOnly>Shekhinah</CustomFloorOnly>
  <!-- other stuff -->
</Stage>
```

use CustomExceptFloor or CustomFloorOnly to restrict your stage from/to being played on custom floors not included in the default values of SephirahType